

Harness 工程在机构投票应用中的探索实践

一、工作开展背景与目的

1.1 工作背景

- 机构投票应用为存量项目，代码规模较大、业务逻辑复杂，传统人工开发模式下需求理解成本高、质量保障依赖个人经验
- Harness 工程简介：一套以 AI Agent 为核心的全流程开发框架，通过多个独立 Agent 协作，覆盖需求分析、编码开发、测试验证、交付提交的完整软件交付链路
- Harness 工程的核心特点：
 - 全流程闭环：需求→开发→测试→交付，各阶段设置独立审查门禁，不通过则回退重修
 - 多 Agent 协作监督：开发与审查由不同 Agent 独立执行，互不共享上下文，从机制上保证审查客观性
 - 资产可复用：Agent、技能、命令均可跨项目迁移，边际成本递减
 - 为什么引入 Harness：传统开发质量高度依赖个人能力与经验，难以标准化；AI 大模型虽有代码生成能力，但缺乏工程化流程约束，容易出现"模型跳过关键步骤"的问题。Harness 将 AI

能力嵌入标准化流程中，实现"AI 执行 + 流程约束 + 独立审查"三位一体的质量保障模式

1.2 工作目标

1. 验证 Harness 全流程开发框架在存量项目改造中的可行性
2. 建立从需求到交付的标准化质量门禁闭环
3. 沉淀可复用的工程资产，为后续项目推广奠定基础

1.3 工作范围

- 项目范围：机构投票应用（QFII 投票）的需求分析、功能开发、集成测试全流程
- 流程范围：覆盖需求与设计、TDD 开发、集成验证、交付提交四个阶段
- 参与角色：AI Agent 集群（需求分析、审查、开发、测试） + 人工决策与验收

二、核心方法论：四阶段质量门禁闭环

2.1 总体流程设计

- 流程驱动而非结果驱动：每个阶段均设置独立的交付物标准和审查

Agent，前一阶段审查通过方可进入下一阶段，不通过则回退重修——这与传统"AI生成代码然后人工复查"的本质区别在于，质量检查被前置并强制嵌入流程中

- 开发与审查分离：编写代码的 Agent 和审查代码的 Agent

以全新上下文独立运行，互不共享信息，杜绝"同一模型既当运动员又当裁判"的问题

- 全链路可追溯：需求文档、审查报告、测试方案、测试结果、提交记录形成完整证据链，每一行代码的变更都能追溯到需求条目和

测试用例

- 人工只做关键决策：人工仅在审查报告评审、流程节点批准等关键决策点介入，日常开发工作由 AI Agent 自主完成

2.2 第一阶段：需求与设计 —— 写代码前把需求说清楚

- 通过代码逆向分析，从存量代码中提取业务需求规格
- 独立审查 Agent 对需求文档进行系统性质量审计（覆盖完整性、可测性、安全性等 10 个维度）
- 交付物：需求规格文档 + 审查报告

2.3 第二阶段：TDD 开发 —— 先写测试，再写代码

- RED 阶段：AI Agent 基于需求规格编写完整单元测试（此时代码尚未实现，测试全部失败）
- RED 审查：独立审查 Agent 检查测试覆盖度、边界情况、反模式
- GREEN 阶段：AI Agent 编写实现代码，迭代至全部测试通过
- 硬性门禁：单元测试覆盖率 $\geq 80\%$
- 交付物：单元测试方案 + 审查报告 + 实现代码

2.4 第三阶段：集成验证 —— 在真实环境中验证业务完整性

- AI Agent 编写端到端集成测试方案，覆盖跨模块业务流程
- 独立审查 Agent 检查方案完整性（场景覆盖、业务规则、边界情况）
- 通过浏览器自动化执行冒烟测试，验证真实用户操作路径
- 问题发现后进入"补测试 → 修代码 → 重跑全量"的修复循环
- 交付物：集成测试方案 + 审查报告 + 执行报告

2.5 第四阶段：交付 —— 标准化、可追溯

- 质量门禁检查：单测全绿 + 集成测试全绿 + 覆盖率达标
- 规范化提交：遵循 Conventional Commits 规范，提交信息可追溯

- 交付物归档：全流程文档、审查报告、测试报告统一留存

三、独立审查机制：多 Agent 互相监督

3.1 审查 Agent 体系设计

- 设置三类独立审查 Agent：需求审查（Spec Auditor）、单测审查（RED Auditor）、集成测试方案审查（Integration Test Plan

Auditor)

- 每个审查 Agent 以全新上下文独立运行，不与开发 Agent 共享信息，确保审查的客观性
- 审查维度标准化，每个 Agent 均有明确的审查清单和评级标准（A/B/C/D 四级）

3.2 问题分级与闭环机制

- 问题按严重程度分为 P0（阻断）、P1（严重）、P2（建议）三级
- 存在 P0/P1 问题则流程回退，修复后重新审查，直至通过

四、实践成效

4.1 实践路径与困难应对

4.1.1 困难一：逆向工程——历史项目需求文档不完整

- 问题：存量项目运营多年，原始需求文档存在缺漏、描述不准确、与代码实际行为不一致等问题
- 解决方案：开发"需求逆向提取"命令，从代码中反推业务规则、校验逻辑、交互流程，形成需求规格初稿；再通过需求审查 Agent

逐条审计，由人工确认补充，补齐缺失信息

4.1.2 困难二：内网算力有限，工具需适配工程化场景

- 问题：Harness 流程要求 AI 模型长时间不间断循环运行，而内网环境中高性能 GPU 资源紧张；且部分开源 AI 编程工具在内网环境下部署复杂、不易集成
- 解决方案：采用国产 AI 编码工具 Qoder，将内网生成的需求拿到外网环境中使用外部算力资源进行开发

4.1.3 困难三：代码地图——防止上下文溢出

- 问题：项目代码量庞大，若一次性将所有代码加载到 AI 上下文，会导致响应质量下降甚至超限
- 解决方案：引入代码知识图谱（Graphify），将代码结构化为节点关系图。AI Agent

按需查询相关模块，精准加载代码片段，避免无关代码干扰决策

4.2 资源投入统计

指标	数据
模型	Qwen 3.7 Max / DeepSeek-V4-Pro
Token 消耗	DeepSeek-V4-Pro 调用约 5.1 亿 token；Qwen 3.7 Max 约 9,000 credits
周期	历时 3 周
人力投入	5 人 / 天（主要集中在关键决策点审查与需求确认）

4.3 质量成效

- 需求文档经独立审查后，歧义项和缺失项大幅减少
- 单元测试覆盖率达到 80% 门禁要求，边界情况和异常路径得到系统覆盖
- 集成测试方案覆盖了跨模块业务流程和多种异常场景

4.4 资产沉淀

- 形成可复用的工程资产包（4 类审查 Agent、3 个开发技能、2 个流程命令）
- 建立了一套从存量项目逆向提取需求的标准流程

4.5 局限性

- Agent 通用性待验证：当前审查 Agent 的审查维度基于本项目实际需求设计，推广至其他类型项目时审查标准需持续优化调整
- 人工介入环节偏多：探索阶段，部分流程环节（如需求确认、审查报告评估、浏览器手动登录）仍需较多人工参与，尚未达到框架

预期的"全流程无人值守"目标

五、后续规划与展望

5.1 集成测试自动化推进

- 推进浏览器自动化技能的完善，实现集成测试的全自动执行与报告生成，减少人工操作

5.2 存量项目进一步探索

- 在当前项目中增加新的业务模块，模拟存量项目场景，验证 Harness 方法论在新模块开发与老模块改造中的适用性和效果差异
- 重点考察：现有审查 Agent 是否适用新模块、需求逆向提取在新模块中是否依然有效

5.3 持续优化方向

- 根据实践反馈持续优化 Agent 审查维度与流程效率

- 探索更多 AI Agent 协作模式（如多 Agent 并行开发、对抗式审查）